



Московский государственный университет имени
М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра автоматизации систем вычислительных комплексов

Рогинко Алла Дмитриевна

**Исследование и реализация жадных
алгоритмов
для задачи распределения таблиц
классификации
по стадиям конвейера СПУ**

Курсовая работа

Научный руководитель:
Доцент, к.ф.-м.н.
Капитонова Алла Петровна
Научный консультант:
Никифоров Никита Игоревич

Москва, 2026

Аннотация

Исследование и реализация жадных алгоритмов
для задачи распределения таблиц классификации
по стадиям конвейера СПУ

Рогинко Алла Дмитриевна

Рассматривается задача размещения логических match-action таблиц P4 по стадиям конвейера архитектуры RMT при учёте графа зависимостей таблиц и ограничений аппаратных ресурсов. Проводится обзор жадных эвристик (FFD, FFL, FFLS, MCF), описывается декомпозиция задачи на этапы топологической упорядоченности, проверки ресурсов и поиска размещения. Представлены программная реализация многопроходного варианта FFD с поддержкой горизонтального разрезания таблиц и эталонной реализации FFLS, а также методика экспериментального сравнения по числу занятых стадий и времени работы на синтетических конфигурациях. Сформулированы выводы о применимости рассмотренных подходов при различных размерах и плотности зависимостей графа.

Abstract

Содержание

1	Введение	5
2	Предметная область	6
2.1	Сетевые процессорные устройства	6
2.2	Архитектура RMT	6
2.3	Программа обработки пакетов и язык P4	7
2.4	Граф зависимостей и виды зависимостей внутри таблиц P4	7
3	Цель работы и постановка задачи	9
3.1	Актуальность работы	9
3.2	Цель работы	9
3.3	Постановка задачи	9
4	Обзор существующих алгоритмов	11
4.1	NP-трудность задачи размещения таблиц	11
4.2	Точные методы: целочисленное линейное программирование (ILP)	11
4.3	Жадные эвристики	11
4.4	Выводы по обзору	12
5	Реализованные алгоритмы	14
5.1	Вычисление потребности в блоках памяти	14
5.2	Горизонтальное разрезание таблиц	15
5.3	Архитектура программной реализации	15
5.3.1	Общие элементы входа, ресурсного учёта и графа зависи- мостей	16
5.3.2	Размещение фрагмента и общий предикат допустимости ста- дии	16
5.3.3	Реализация многопроходного FFD	17
5.3.4	Реализация FFLS	18
5.3.5	Вычисление уровней	18
6	Экспериментальное исследование	20
6.1	Методика экспериментального исследования	20

6.1.1	Цели эксперимента	20
6.1.2	Исходные данные	20
6.1.3	Параметры эксперимента	21
6.1.4	Обработка результатов	21
6.2	Анализ полученных результатов	21
6.2.1	Результаты сравнения по времени выполнения	21
6.2.2	Результаты сравнения по количеству стадий	23
6.2.3	Гистограммы с усами	24
6.3	Погрешности	25
7	Заключение	26
	Список литературы	27

1 Введение

В современных сетях возникает необходимость в гибкой обработке пакетов. Необходимо быстро внедрять новые протоколы и сервисы без замены оборудования. Решением этого вопроса является использование программируемых сетевых устройств с архитектурой RMT с языком описания пакетной обработки P4. Такие устройства реализуют конвейерную обработку, состоящую из последовательных match-action стадий.

При компиляции P4-программы для архитектуры RMT нужно распределить логические таблицы по стадиям конвейера. От этого решения зависят число задействованных стадий и, следовательно, задержка обработки пакета. Для успешного размещения следует учитывать ограничения архитектуры аппаратуры и порядок применения таблиц, заданный графом зависимостей (Table Dependency Graph). На практике стремятся минимизировать число занятых стадий, поскольку оно непосредственно определяет задержку обработки пакета.

В данной работе рассматриваются жадные алгоритмы решения задачи размещения, анализируется их эффективность и быстрота выполнения, и предлагается модификация с поддержкой горизонтального разрезания таблиц.

2 Предметная область

2.1 Сетевые процессорные устройства

Сетевые процессорные устройства (СПУ) — это специализированные системы, предназначенные для обработки сетевого трафика на высоких скоростях. Проще говоря, такие устройства должны очень быстро принимать пакеты, анализировать их и принимать решение о дальнейшей обработке. В отличие от традиционных сетевых устройств, где логика работы часто жёстко зафиксирована на уровне аппаратуры, программируемые СПУ позволяют изменять алгоритмы обработки пакетов и адаптировать их под разные задачи. Отличительной чертой современных СПУ является наличие программируемой логики обработки пакетов, что позволяет адаптировать устройство под новые протоколы и сетевые функции без замены аппаратного обеспечения.

2.2 Архитектура RMT

Одной из архитектур эталонной для построения высокопроизводительных программируемых сетевых устройств является RMT (Reconfigurable Match Tables). Основная идея этой архитектуры заключается в использовании набора настраиваемых таблиц сопоставления (match tables), которые применяются на разных стадиях конвейера обработки пакетов. На каждой стадии на основе выбранного ключа выполняется поиск в одной или нескольких таблицах, где правила сопоставления определяют, какие действия должны быть выполнены над пакетом. Эти действия могут включать изменение заголовков, выбор следующей стадии обработки или передачу пакета на выходной интерфейс. Благодаря такой структуре архитектура остаётся гибкой и позволяет реализовывать различные алгоритмы обработки сетевого трафика.

Match-action стадии — последовательность одинаковых стадий. Каждая стадия содержит:

1. Таблицы точного поиска, реализованные на SRAM.
2. Таблицы тернарного и префиксного поиска, реализованные на TCAM.

3. Кроссбары для подачи произвольных полей из PHV на вход таблиц.
4. Action-блоки для параллельного изменения полей PHV.

2.3 Программа обработки пакетов и язык P4

Для описания поведения программируемого коммутатора используется язык P4 (Programming Protocol-independent Packet Processors). Программа на P4 определяет набор логических match-action таблиц, каждая из которых включает набор полей заголовка, по которым производится поиск и список возможных действий для них. P4 является аппаратно-независимым языком: одна и та же программа может компилироваться для различных устройств.

2.4 Граф зависимостей и виды зависимостей внутри таблиц P4

При компиляции P4-программы в конвейер RMT строится граф зависимостей таблиц (Table Dependency Graph, TDG). Вершинами графа являются логические таблицы, а рёбрами — зависимости между таблицами. Корректное размещение таблиц по стадиям конвейера должно удовлетворять всем видам зависимостей, иначе обработка пакетов будет нарушена. Выделяют четыре основных типа зависимостей:

Match-зависимость возникает, когда таблица A изменяет поле, а таблица B выполняет поиск по этому полю. Эта зависимость соответствует классической зависимости «чтение после записи». Для корректной работы необходимо, чтобы A полностью завершила выполнение своих действий до того, как B начнёт поиск. Поэтому, таблицы с match-зависимостью не могут располагаться в одной стадии конвейера.

Action-зависимость появляется, когда две таблицы A и B изменяют одно и то же поле, и важен результат, полученный в таблице B . При размещении допускается нахождение A и B в одной стадии.

Successor-зависимость заключается в следующем: выполнение таблицы B зависит от результата таблицы A . Таблицы с successor-зависимостью могут размещаться в одной стадии.

Обратная match-зависимость возникает, когда таблица A выполняет поиск по полю, которое впоследствии изменяется таблицей B . Также требует строгого разделения стадий: A должна быть размещена в стадии, строго меньшей, чем B .

3 Цель работы и постановка задачи

3.1 Актуальность работы

Рост скорости сетевого трафика и требование быстро менять поведение коммутаторов без замены оборудования делают актуальной программируемую сетевую обработку. Компиляция Р4-программы в конвейер RMT включает этап размещения логических таблиц по стадиям при ограничениях памяти и графе зависимостей. Качество этого этапа влияет на число занятых стадий и задержку обработки пакета, поэтому исследование эвристик размещения и их реализации для типовых ограничений TDG сохраняет практический интерес для разработчиков компиляторов и сетевого оборудования.

3.2 Цель работы

Целью данной работы является исследование жадных алгоритмов решения задачи размещения логических таблиц классификации по стадиям конвейера RMT-коммутатора, а также разработка программной реализации этих алгоритмов для оценки ресурсных затрат Р4-программ.

3.3 Постановка задачи

Формально задача размещения таблиц классификации может быть сформулирована следующим образом.

Дано:

1. Множество логических таблиц $T = \{t_1, t_2, \dots, t_n\}$, каждая из которых характеризуется:
 - типом сопоставления $\text{match_type}(t_i) \in \{\text{exact}, \text{ternary}, \text{lpm}\}$;
 - шириной ключа $w(t_i)$ (в битах);
 - максимальным количеством записей $e(t_i)$;
 - списком зависимостей от других таблиц с указанием типа зависимости $\text{dep_type} \in \{\text{match}, \text{action}, \text{successor}, \text{reverse_match}\}$.

2. Описание аппаратной архитектуры RMT:

- количество match-action стадий S_{total} ;
- количество блоков SRAM на стадию B_{sram} , ширина блока W_{sram} (бит), глубина блока D_{sram} (количество записей);
- количество блоков TCAM на стадию B_{tcam} , ширина блока W_{tcam} (бит), глубина блока D_{tcam} (количество записей).

Требуется: Построить схему размещения, которая каждой таблице (или каждому фрагменту таблицы при горизонтальном разрезании) ставит в соответствие номер стадии конвейера, такое, что:

1. Выполнены все аппаратные ограничения.
2. Соблюдены все типы зависимостей между таблицами.
3. Количество использованных стадий минимально.

4 Обзор существующих алгоритмов

4.1 NP-трудность задачи размещения таблиц

Задача размещения логических таблиц классификации по стадиям конвейера RMT является вычислительно сложной. В работе [?] доказано, что эта задача является NP-трудной. Более того, авторы показывают, что для ряда моделей не существует алгоритма, который решал бы её точно за полиномиальное время.

Следовательно, для практического применения при значительных объёмах входных данных (десятки и сотни таблиц) необходимо использовать приближённые методы, в частности, жадные эвристики.

4.2 Точные методы: целочисленное линейное программирование (ILP)

Точное решение задачи размещения может быть получено с помощью целочисленного линейного программирования (Integer Linear Programming, ILP). В работе [?] предложена ILP-модель, включающая:

- переменные, указывающие, в каких стадиях размещены таблицы;
- ограничения, гарантирующие соблюдение ёмкости памяти и зависимостей;
- целевую функцию, минимизирующую количество использованных стадий.

ILP позволяет найти оптимальное размещение, однако время его работы экспоненциально зависит от числа таблиц. Эксперименты из [?] показывают, что для программы с 24 логическими таблицами ILP-решатель затрачивает около 6300 секунд на поиск оптимума. Для более крупных программ время становится неприемлемым для практического использования, особенно при необходимости перекомпиляции в реальном времени.

4.3 Жадные эвристики

В связи с NP-трудностью задачи для практических применений разработаны жадные эвристики, которые работают за полиномиальное время и обеспе-

чивают приемлемое качество размещения. В литературе [?] выделяют четыре основных типа эвристик:

FFD (First Fit Decreasing) – классический алгоритм упаковки. Таблицы сортируются по убыванию количества требуемых блоков памяти. Затем каждая таблица размещается в первую стадию, в которой для неё достаточно свободной памяти. FFD не учитывает топологический порядок зависимостей, что может приводить к невозможности разместить корректно все таблицы при наличии match-зависимостей.

FFL (First Fit by Level) – перед сортировкой вычисляются уровни таблиц: уровень таблицы равен длине самого длинного пути в графе зависимостей от любого источника. Таблицы сортируются по возрастанию уровня, а внутри уровня – в произвольном порядке. Это гарантирует, что предки обрабатываются раньше потомков, однако из-за произвольного порядка внутри уровня это может приводить к неэффективному использованию памяти на одной стадии.

FFLS (First Fit by Level and Size) – модификация FFL, в которой внутри одного уровня таблицы дополнительно сортируются по убыванию размера. Данный подход сочетает преимущества топологического порядка и плотной упаковки.

MCF (Most Constrained First) – таблицы сортируются по степени «ограниченности»: в первую очередь обрабатываются таблицы, которые могут быть размещены только в определённом типе памяти (например, тернарные – только в ТСАМ) или имеют наибольшую ширину ключа.

4.4 Выводы по обзору

Таким образом, задача размещения таблиц является NP-трудной. Точные методы (ILP) обеспечивают оптимальность, но неприменимы для больших программ из-за экспоненциального времени работы. Жадные эвристики работают быстрее. Однако классические жадные эвристики имеют недостаток – они могут

не найти размещение для всех таблиц из-за однопроходности. Поэтому актуальной является разработка многопроходных и гибридных подходов. В последующих разделах будет представлена реализация многопроходного FFD и FFLS, а также результаты их экспериментального сравнения по времени работы и количеству использованных стадий.

5 Реализованные алгоритмы

В данной главе описываются три основных механизма, реализованных в рамках курсовой работы: горизонтальное разрезание таблиц, многопроходный алгоритм FFD (First Fit Decreasing) и алгоритм FFLS (First Fit by Level and Size). Первый является вспомогательным и применяется, когда таблица не помещается целиком в одну стадию. Второй представляет собой авторскую модификацию классического FFD, позволяющую избегать тупиков за счёт многопроходного цикла. Третий — известная из литературы эвристика, реализованная для сравнения. Все алгоритмы учитывают аппаратные ограничения, а также типы зависимостей между таблицами.

5.1 Вычисление потребности в блоках памяти

Перед размещением для каждой логической таблицы t необходимо определить, сколько физических блоков памяти (SRAM или TCAM) потребуется для хранения всех её записей.

Обозначим:

- $w(t)$ — ширина ключа таблицы (бит);
- $e(t)$ — максимальное количество записей в таблице;
- W_{mem} — ширина одного блока памяти (бит);
- D_{mem} — глубина блока (количество ячеек, или записей, которое может хранить один блок).

Для точного поиска (exact) используется SRAM, для тернарного и префиксного (ternary, lpm) — TCAM. Количество блоков, необходимых для таблицы t , вычисляется по формуле:

$$\text{blocks}(t) = \left\lceil \frac{w(t)}{W_{\text{mem}}} \right\rceil \times \left\lceil \frac{e(t)}{D_{\text{mem}}} \right\rceil.$$

где $\lceil w(t)/W_{\text{mem}} \rceil$ — определяет, сколько блоков нужно для хранения одной записи (если ключ шире блока, он распределяется по нескольким блокам).

Второй множитель — сколько таких «полосок» требуется для размещения всех записей. Полученное значение $\text{blocks}(t)$ используется во всех последующих алгоритмах как «размер» таблицы.

5.2 Горизонтальное разрезание таблиц

Горизонтальное разрезание применяется, когда $\text{blocks}(t)$ превышает максимальное количество блоков C , которое может быть выделено в одной стадии для данного типа памяти. Таблица разбивается на $k = \lceil \text{blocks}(t)/C \rceil$ фрагментов. Размер первого фрагмента равен C , последнего — $\text{blocks}(t) - (k - 1)C$, все промежуточные — также C . Фрагменты размещаются в последовательных стадиях конвейера, начиная с некоторой начальной стадии s , которая определяется динамически в процессе работы алгоритмов размещения.

Условия корректности разрезания:

1. Фрагменты одной таблицы должны занимать стадии с номерами $s, s + 1, \dots, s + k - 1$ без пропусков. Это требование обусловлено аппаратной реализацией RMT: после неудачного поиска в стадии i пакет переходит в стадию $i + 1$, а не в произвольную.
2. Если таблица A является предком для таблицы B по `match`- или `reverse_match`-зависимости и хотя бы одна из них разрезана, то должно выполняться

$$\max_{\text{фрагменты } A} \text{stage} < \min_{\text{фрагменты } B} \text{stage}.$$

Если разрезана только B (потомок), то достаточно $\text{stage}(A) < \min \text{stage}(B)$; если разрезана только A — $\max \text{stage}(A) < \text{stage}(B)$.

5.3 Архитектура программной реализации

Разработанная реализация алгоритмов размещения представляет собой два исполнимых скрипта на Python 3. Оба используют один и тот же способ кодирования входных данных. Однако, различаются порядок обхода таблиц и логикой размещения.

5.3.1 Общие элементы входа, ресурсного учёта и графа зависимостей

Описание одной материальной стадии читается из JSON файла конфигурации. Список логических таблиц находится также в JSON. Чтение данных файлов реализовано соответственно в функциях `read_hardware` и `read_tables`.

Для каждой таблицы перед размещением вызывается единая функция `calc_blocks_needed` для вычисления потребности в блоках памяти по формуле из подраздела *Вычисление потребности в блоках памяти* при подстановке параметров железа.

Функция построения графа зависимостей описывается двумя словарями смежности: `depends_on` — по ключу имени таблицы хранится список пар (предок, тип) для ограничений на размещаемую таблицу; `dependents` — симметрично список пар (потомок, тип) для уже размещённых узлов ниже по конвейеру. Такое дублирование упрощает проверку и предков, и уже зафиксированных потомков при выборе индекса стадии. Она реализована как `build_dependency_graph`

Результат размещения хранится в двух основных структурах. Список `stages` задаёт упорядоченные вершины физических стадий: каждый элемент — счётчики занятых блоков SRAM и TCAM (`sram`, `tcam`). Словарь `table_to_fragments` сопоставляет имени таблицы список записей номер стадии, количество блоков — по ним восстанавливается фактическое разбиение большой таблицы на несколько стадий.

5.3.2 Размещение фрагмента и общий предикат допустимости стадии

В обоих скриптах размещение одной порции блоков выполняется в цикле: перебираются существующие стадии в порядке возрастания номера и первая стадия, для которой остаток свободных блоков выбранного типа памяти положителен и выполняются условия TDG относительно уже зафиксированных предков и потомков, пополняется на величину $\min(\text{остаток таблицы, свободно на стадии})$. Если ни одну существующую стадию нельзя расширить, создаётся новая запись стадии в конце списка, и блоки добавляются в неё тем же правилом. Эта логика реализована в функции `try_place_fragment`.

Проверка зависимостей локализована в функциях `can_place_fragment`. Для каждого размещённого предка сравнивается максимальный индекс стадии

среди его фрагментов с индексом кандидата. Так совмещаются рассмотренные в теоретической части четыре вида ограничений.

5.3.3 Реализация многопроходного FFD

Классический FFD сортирует таблицы по убыванию $\text{blocks}(t)$ и однократно размещает их в первую подходящую стадию. Его недостаток — возможные тупики при наличии match-зависимостей: если большая таблица-потомок обрабатывается раньше маленького предка, то предок впоследствии не может быть размещён.

Предлагаемый многопроходный FFD устраняет эту проблему за счёт повторных проходов по ещё не размещённым таблицам.

Центральная функция `ffd_mapping_with_splitting` упорядочивает вход по убыванию `blocks_total` и ведёт список `remaining` ещё не размещённых таблиц. На каждом проходе по этому списку перебираются все оставшиеся записи в фиксированном порядке; таблица пропускается, если не все имена её предшественников уже помечены во множестве `placed_tables`. Иначе вызывается `try_place_table`, которая открывает внутренний цикл пополнения блоков до полного счёта и при успешном завершении переносит таблицу из `remaining` во множество размещённых. Если при размещении выясняется противоречие при создании новой стадии, функция завершает работу всего процесса с частичным результатом. Если за полный проход по списку не было ни одного успеха, многопроходный алгоритм фиксирует тупик и оставляет недостроенный результат. Точка входа для пользователя выполняет чтение JSON, печать отчёта по стадиям и явную проверку всех пар предок–потомок по сохранённым фрагментам.

Algorithm 1 Многопроходный FFD с разрезанием

Input: Множество таблиц T , ёмкость стадии C , зависимости

Output: Размещение F (фрагменты по стадиям) или ошибка

```
1: вычислить  $\text{blocks}(t)$  для всех  $t \in T$ 
2: отсортировать  $T$  по убыванию  $\text{blocks}(t)$ 
3:  $S \leftarrow \emptyset$  (список стадий),  $\text{placed} \leftarrow \emptyset$ 
4: while  $T \setminus \text{placed} \neq \emptyset$  do
5:    $\text{progress} \leftarrow \text{False}$ 
6:   for each  $t \in T \setminus \text{placed}$  в порядке сортировки do
7:     if  $\text{pred}(t) \subseteq \text{placed}$  then
8:       попытаться разместить все фрагменты  $t$  в существующих или новых
       стадиях (First Fit с последовательными номерами)
9:       if размещение удалось then
10:        добавить фрагменты в  $F$ ,  $\text{placed} \leftarrow \text{placed} \cup \{t\}$ ,  $\text{progress} \leftarrow \text{True}$ 
11:       end if
12:     end if
13:   end for
14:   if  $\neg \text{progress}$  then
15:     return ошибка (тупик)
16:   end if
17: end while
18: return  $F$ 
```

5.3.4 Реализация FFLS

Алгоритм FFLS [?] является однопроходным и основан на топологическом упорядочении таблиц. Он гарантирует, что ни одна таблица не будет обработана раньше своих предков, что исключает тупики, связанные с порядком размещения.

5.3.5 Вычисление уровней

Для каждой таблицы t вычисляется уровень $\text{level}(t)$ как длина самого длинного пути в графе зависимостей от любого источника (таблицы без пред-

ков). В коде эта функция называется `compute_levels`. Формально вычисление уровней выглядит так:

$$\text{level}(t) = \begin{cases} 0, & \text{если } \text{pred}(t) = \emptyset; \\ \max_{p \in \text{pred}(t)} (\text{level}(p) + 1), & \text{иначе.} \end{cases}$$

Для любого ребра зависимости $p \rightarrow t$ выполняется $\text{level}(p) < \text{level}(t)$.

Algorithm 2 FFLS с разрезанием

Require: Множество таблиц T , ёмкость стади C , зависимости

Ensure: Размещение F

- 1: вычислить $\text{blocks}(t)$ и $\text{level}(t)$ для всех t
 - 2: отсортировать T по $(\text{level}(t), -\text{blocks}(t))$ (возрастание уровня, убывание размера)
 - 3: $S \leftarrow \emptyset$ (список стадий)
 - 4: **for** each $t \in T$ в отсортированном порядке **do**
 - 5: разместить фрагменты t методом First Fit (аналогично шагу размещения в многопроходном FFD, но без внешнего цикла по проходам)
 - 6: **if** размещение невозможно **then**
 - 7: **return** ошибка
 - 8: **end if**
 - 9: **end for**
 - 10: **return** F
-

Сначала вычисляются уровни таблиц, затем выполняется сортировка: сначала по возрастанию уровня, потом по убыванию $\text{blocks}(t)$. Далее таблицы обрабатываются однократно в этом порядке. Для каждой таблицы применяется та же процедура размещения с поддержкой разрезания и проверкой зависимостей, что и в многопроходном FFD. Эти действия представлены в функции `ffls_mapping`. В конце `main` выводится такая же агрегированная информация по стадиям и проверка соответствия рёбер TDG.

6 Экспериментальное исследование

В данной главе описывается методика проведения экспериментов, анализируются полученные результаты и формулируются выводы о сравнительной эффективности реализованных алгоритмов.

6.1 Методика экспериментального исследования

6.1.1 Цели эксперимента

Основными целями экспериментального исследования являются:

- сравнение многопроходного FFD и FFLS по времени выполнения;
- сравнение алгоритмов по количеству использованных стадий конвейера;
- оценка влияния количества таблиц и количества записей в таблицах на производительность алгоритмов;

6.1.2 Исходные данные

Для экспериментов были сгенерированы синтетические конфигурации, каждая из которых представляет собой граф зависимостей таблиц (TDG) и параметры таблиц. Конфигурации различались по двум параметрам:

- **Количество таблиц** $N \in \{30, 40, 50, 100, 200, 300\}$.
- **Количество записей в таблице** $E \in \{256, 1024, 4096, 16384\}$ (все таблицы в одной конфигурации имели одинаковое число записей).

Для каждой комбинации (N, E) было подготовлено по 20 различных графов зависимостей, отличающихся структурой (количеством и типами рёбер). Таким образом, общее количество тестовых конфигураций составило $6 \times 4 \times 20 = 480$.

Аппаратная конфигурация (RMT-коммутатор) была зафиксирована в соответствии с описанием V1Model:

- 32 match-action стадии;

- 106 блоков SRAM на стадию (ширина 80 бит, глубина 1024 записи);
- 16 блоков TCAM на стадию (ширина 40 бит, глубина 2048 записи).

6.1.3 Параметры эксперимента

Каждый алгоритм запускался на каждой конфигурации 10 раз. Такое количество прогонов необходимо для уменьшения влияния случайных флуктуаций времени выполнения, связанных с фоновыми процессами операционной системы. Измерялись следующие величины:

- время выполнения алгоритма (в миллисекундах)
- количество использованных стадий конвейера

6.1.4 Обработка результатов

Для каждой комбинации (N, E) и каждого алгоритма по 20 конфигурациям и 10 прогонам вычислялись:

- среднее арифметическое времени выполнения $\bar{T}$;
- минимальное $T_{\min}$ и максимальное $T_{\max}$ значения времени;
- среднее количество стадий $\bar{S}$, а также $S_{\min}$ и $S_{\max}$.

Усреднение по 10 прогонам позволяет уменьшить влияние «шума» – кратковременных задержек, вызванных внешними процессами. Анализ гистограмм с усами даёт представление о разбросе результатов. Все вычисления производились с помощью скриптов на Python с использованием библиотек `pandas` и `numpy`.

6.2 Анализ полученных результатов

6.2.1 Результаты сравнения по времени выполнения

На Рис. 1 представлены линейные графики зависимости среднего времени выполнения от количества записей в таблице для разных количеств таблиц.

Наблюдения:

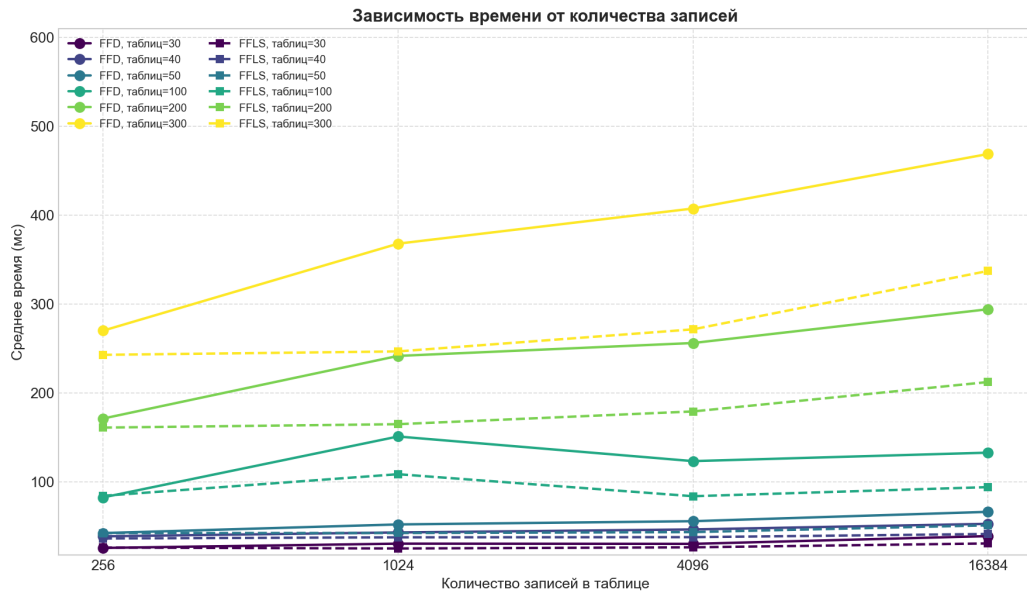


Рис. 1: Зависимость времени выполнения от количества записей

- С ростом количества таблиц время выполнения обоих алгоритмов увеличивается, что ожидаемо.
- Для малого числа таблиц (30, 40, 50) разница между алгоритмами незначительна и может составлять доли миллисекунды; в этих случаях FFD иногда оказывается быстрее, иногда медленнее – это можно объяснить случайными вариациями порядка обработки.
- При $N \geq 100$ преимущество FFLS становится отчётливым. Например, при 300 таблицах и 4096 записях FFLS работает примерно в 2–3 раза быстрее многопроходного FFD.
- Увеличение количества записей (размера таблиц) приводит к росту времени выполнения, особенно заметному для FFD, так как многопроходный алгоритм требует большего числа итераций при разрезании крупных таблиц.

6.2.2 Результаты сравнения по количеству стадий

На Рис. 2 приведены линейные графики зависимости среднего числа стадий от количества записей.

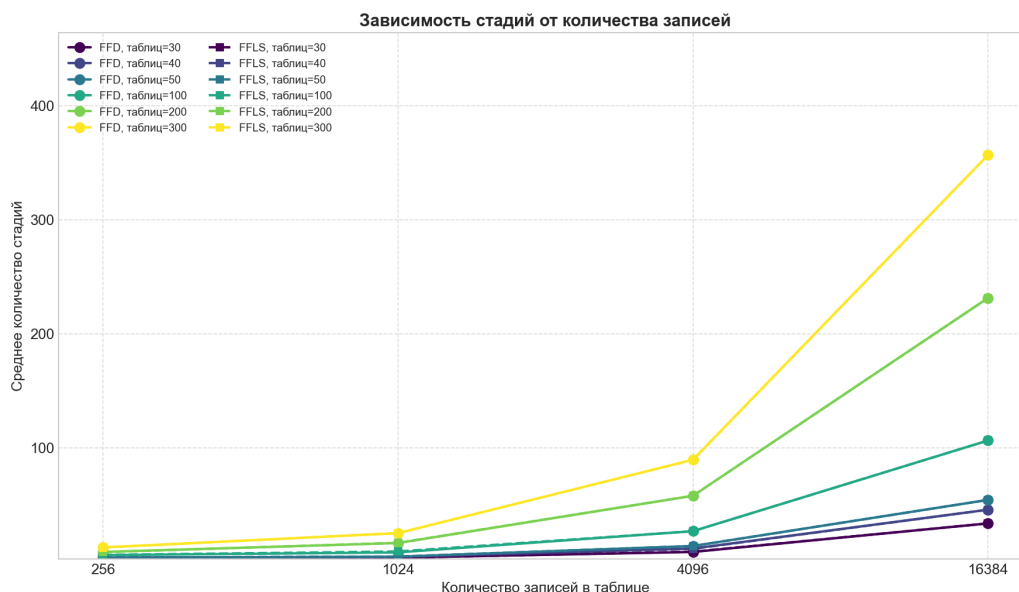


Рис. 2: Зависимость количества стадий от количества записей

Ключевые выводы:

- При фиксированном количестве таблиц увеличение числа записей не сильно влияет на количество стадий. Это объясняется тем, что при разрезании таблицы распределяются по последовательным стадиям, но общее число стадий в основном определяется структурой графа зависимостей, а не абсолютным размером таблиц.
- С ростом количества таблиц от 30 до 300 количество стадий возрастает примерно линейно. FFLS и многопроходный FFD показывают почти одинаковое среднее число стадий (отличие не превышает 5%).
- На малых количествах таблиц (30–50) FFD иногда даёт чуть более плотную упаковку, поскольку FFD всегда пытается поместить крупные таблицы первыми, он может уменьшить число стадий на небольших графах. Однако при $N \geq 100$ различия становятся незначительными.

6.2.3 Гистограммы с усами

Для наглядного представления разброса значений были построены гистограммы с усами (min–mean–max).

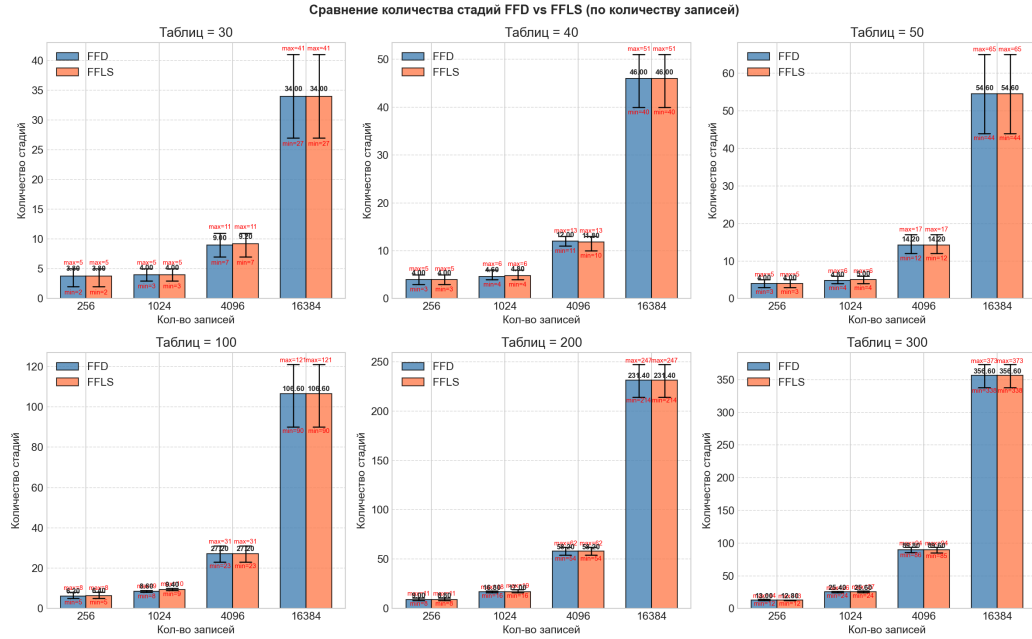


Рис. 3: Гистограмма сравнения стадий по количеству записей

Анализ гистограмм позволяет сделать следующие заключения:

- Разброс значений (разница между минимальным и максимальным числом стадий) составляет от 2 до 5 стадий в зависимости от сложности графа. Это обусловлено различной структурой зависимостей в разных конфигурациях.
- Выбросы встречаются в конфигурациях с длинными цепочками match-зависимостей, где таблицы вынуждены занимать разные стадии.
- Минимальные значения соответствуют конфигурациям, в которых большинство зависимостей являются successor или action, что позволяет размещать таблицы в одном или двух уровнях стадий.

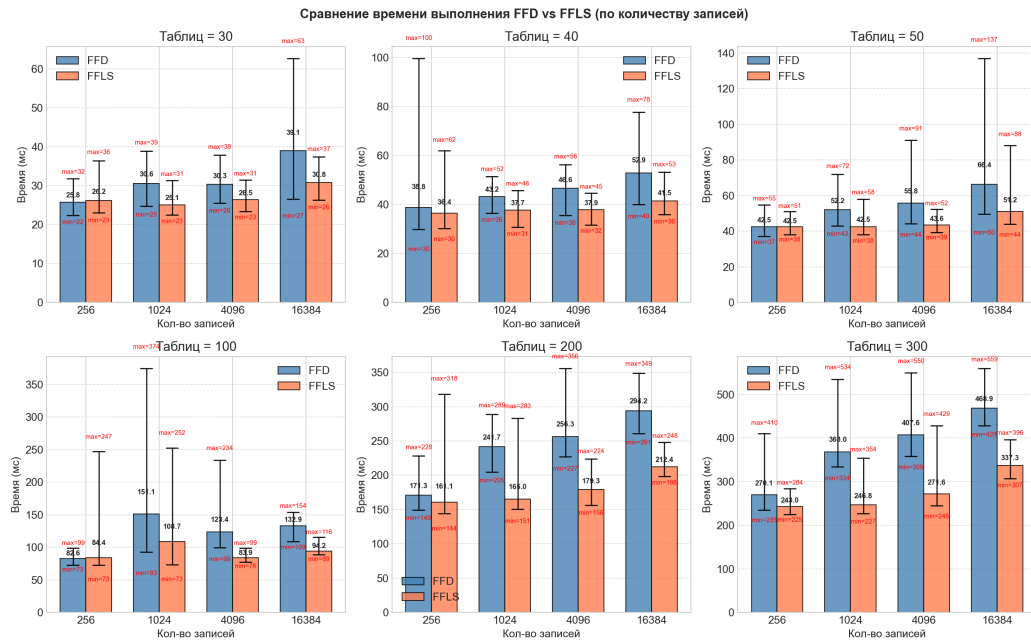


Рис. 4: Гистограмма сравнения времени по количеству записей

6.3 Погрешности

Как было отмечено в методике, время выполнения алгоритмов может подвергаться кратковременным задержкам из-за фоновых процессов операционной системы. Эти задержки проявляются в виде отдельных выбросов (например, время одного прогона может быть в 1.5–2 раза выше среднего). Для снижения их влияния используется усреднение по 10 прогонам. Выбросы не удаляются, так как они являются частью реальных условий исполнения на обычном компьютере; их учёт позволяет получить более консервативную оценку производительности.

7 Заключение

В рамках данной курсовой работы была исследована задача размещения логических таблиц классификации по стадиям конвейера программируемых РМТ-коммутаторов. Актуальность работы обусловлена необходимостью эффективной компиляции Р4-программ для высокопроизводительного сетевого оборудования без замены аппаратной части.

Был проведён анализ предметной области. Формализована задача размещения и перечислены аппаратные ограничения. Рассмотрен обзор существующих алгоритмов, показана NP-трудность задачи и обосновано применение жадных эвристик.

Разработанная реализация двух алгоритмов представлена в виде двух программ на языке Python 3: многопроходного FFD, являющегося авторская модификацией теоретического алгоритма и классического FFLS. Подключена поддержка горизонтального разрезания таблиц и учёт всех четырёх типов зависимостей.

Проведено экспериментальное исследование на синтетических конфигурациях с различным количеством таблиц и записей. Установлено, что оба алгоритма обеспечивают размещения данных конфигураций. FFLS существенно превосходит многопроходный FFD по времени выполнения, а по количеству стадий различия не превышают 5%. Результаты визуализированы в виде линейных графиков и гистограмм с усами. Показано, что увеличение числа записей слабо влияет на число стадий, но заметно увеличивает время работы, особенно для FFD.

Таким образом, цель работы — исследование жадных алгоритмов размещения таблиц классификации и их программная реализация — достигнута. Практическая значимость работы заключается в создании открытого инструмента для оценки реализуемости Р4-программ на целевых РМТ-коммутаторах.

Список литературы

- [1] P4 Language Specification. — P4.org. — дата обращения: 11.05.2026. <https://p4.org/specs/>.
- [2] P4: Programming Protocol-Independent Packet Processors / Pat Bosshart, Dan Daly, Glen Gibb et al. // *ACM SIGCOMM Computer Communication Review*. — 2014. — Vol. 44, no. 3. — Pp. 87–95.
- [3] p4c: reference compiler for the P4 language. — GitHub. — дата обращения: 11.05.2026. <https://github.com/p4lang/p4c>.